



Southeast University

Introduction to Software Engineering

Wang Lulu, Liao Li

lliao@seu.edu.cn

Outline

Part 0. Overview

Part 1. Software Processes

Part 2. Modeling

❖ **Chap 7. Understanding Requirements**

❖ **Chap 8-10. Requirements Modeling**

❖ **Chap 11. Design Concepts**

❖ **Chap 12-14. Design Methods**

Part 3. Quality Assurance

Part 4. Software Project Management

Chap 7. Understanding Requirements

❖ System Engineering

❖ Requirements Engineering

- Inception（起始过程）
- Elicitation（导出过程）
- Elaboration（精化过程）
- Negotiation（协商过程）
- Specification（规格说明）
- Validation（验证过程）
- Requirements management（管理过程）

System Engineering

❖ Elements of a computer-based system

- Software
- Hardware
- People
- Database
- Documentation
- Procedures

❖ Systems

- A hierarchy of macro-elements
宏元素的层次结构

Computer based system

- ❖ 所谓基于计算机的系统是指：通过处理信息来完成某些预定义目标而组织在一起的元素的组合
- ❖ 对于用户而言有意义的是可以达到预期目标的系统(完整的软硬件解决方案)而不是单一软件
- ❖ 组成基于计算机系统的元素主要有：软件、硬件、人员、数据库(及其它系统软件)、文档和规程(**Procedure**)

Computer based system

❖ 基于计算机的系统示例

❖ 校园一卡通

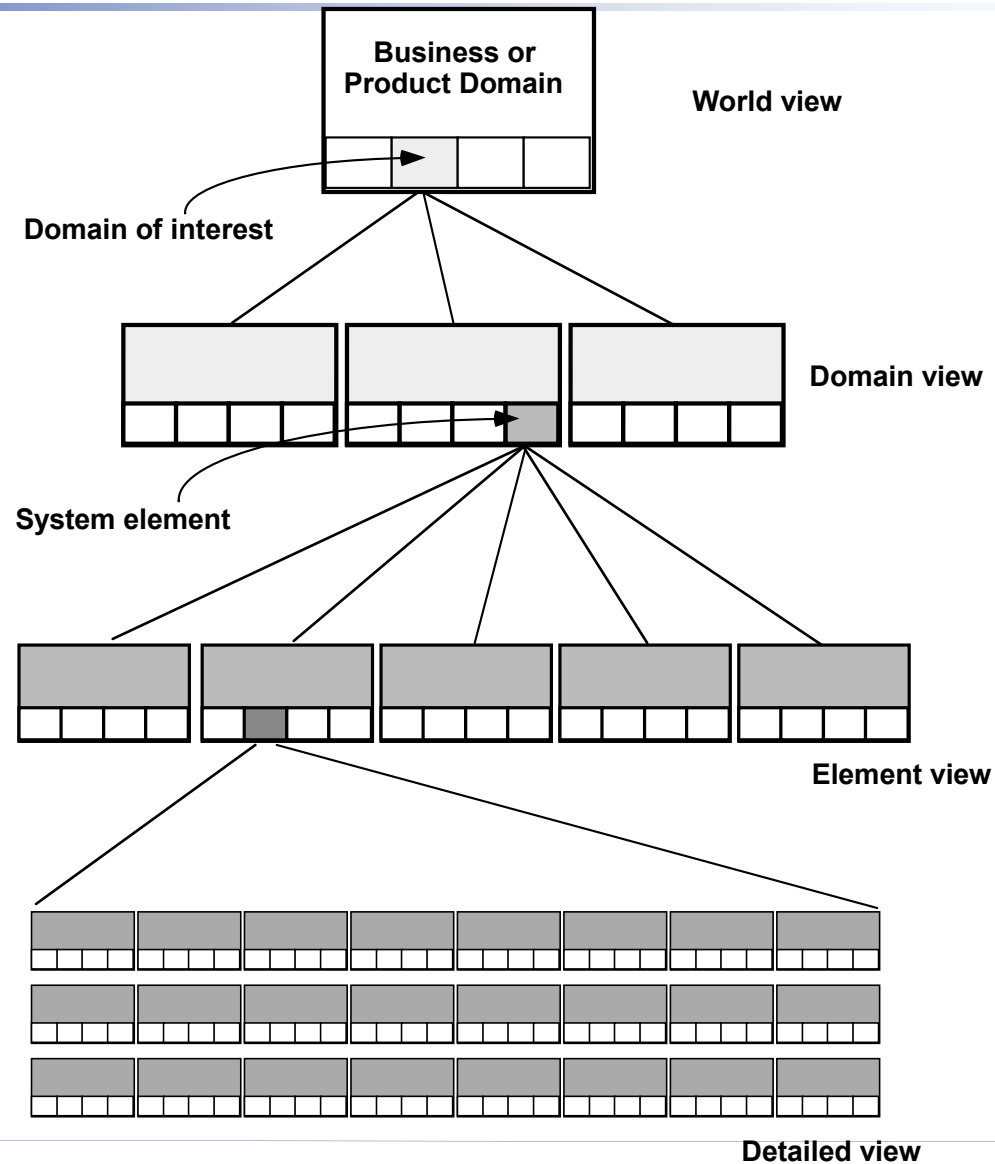
- 硬件设备：服务器、刷卡器、一卡通网络、IC卡
- 软件系统：结算软件、银行接口、查询系统...
- 规程/人员：设计与一卡通相关办理、消费、挂失、充值、结算等制度和流程，设置相应的岗位和职责

Computer based system

❖ 系统的层次结构

- 任何系统都处在一个更大的系统之中，形成系统的层次结构
- 基于计算机的系统可呈现一个层次结构，例如：
 - 校园一卡通系统包括基础网络、结算系统、银行接口系统、消费终端等子系统
 - 一卡通系统处于整个学校系统(教务、财务、学工...)之中

The hierarchy



城市大脑：AI开放创新平台（AI新基建）

City Brain – Open Innovation Platform of AI (AI New Infrastructure)

智能应用
开源开放
共享平台

赋能行业

交通

市政

商圈

电网

司法

地铁

教育

.....

智能应用
开源开放
共享平台

应用生态

中小微企业

高等院校

研究院所

开发者

.....

多模态
数据资源
智能处理
平台

城市AI能力

视频识别

语音识别

OCR识别

图像识别

城市AI 智能引擎

知识图谱与三维重建引擎

预测与干预引擎

场景识别与推理引擎

事件检测引擎

图像搜索引擎

.....

第三方
算法接入

数据智能
计算平台

边云协同分布式视觉计算平台

算子&编译

并行化计算框架

一站式开发

边云协同

大规模视觉数据计算基础服务平台

流计算

任务调度

模型优化

数据索引

图形计算

数据接入

图像数据

视频数据

市政综治数据

环境数据

互联网数据

电网数据

医院数据

工业数据

农业数据

Requirements Engineering-I

- ❖ **Inception (起始)** —ask a set of questions that establish ...
 - basic understanding of the problem
 - the people who want a solution
 - the nature of the solution that is desired, and
 - the effectiveness of preliminary communication and collaboration between the customer and the developer
- ❖ **Elicitation (导出)** —elicit requirements from all stakeholders
- ❖ **Elaboration (精化)** —create an analysis model that identifies data, function and behavioral requirements
- ❖ **Negotiation (协商)** —agree on a deliverable system that is realistic for developers and customers

Requirements Engineering-II

- ❖ **Specification (规格说明)** —can be any one (or more) of the following:
 - A written document
 - A set of models
 - A formal mathematical
 - A collection of user scenarios (use-cases)
 - A prototype
- ❖ **Validation (验证)** —a review mechanism that looks for
 - errors in content or interpretation
 - areas where clarification may be required
 - missing information
 - inconsistencies (a major problem when large products or systems are engineered)
 - conflicting or unrealistic (unachievable) requirements.
- ❖ **Requirements management**

Requirements Engineering

2. 需求工程

1. 过程活动☆☆

1. 起始——与客户建立初步交流

- 确定利益相关者
- 问题和解决的大致方向

2. 导出——通过各种途径获取用户的需求

- 识别问题
- 提出解决要素
- 协商方法
- 确认放案

3. 精化——创建数据、功能和行为要求的分析模型

- 建立分析模型中的元素
 - 基于场景的元素——用例图
 - 基于类的元素——类图
 - 基于行为的元素——状态图
 - 基于数据流的元素——数据流图

4. 协商——开发者与客户之间就所要交付的系统的功能、性能、交付时间等问题进行估算和协商

- 识别关键的利益相关者
- 确认利益相关者的满足条件
- 协商

5. 规格说明——根据需求调查和需求分析的结果，进一步定义准确无误的产品需求，产生《产品需求规格说明书》

6. 确认——进一步确认规格说明

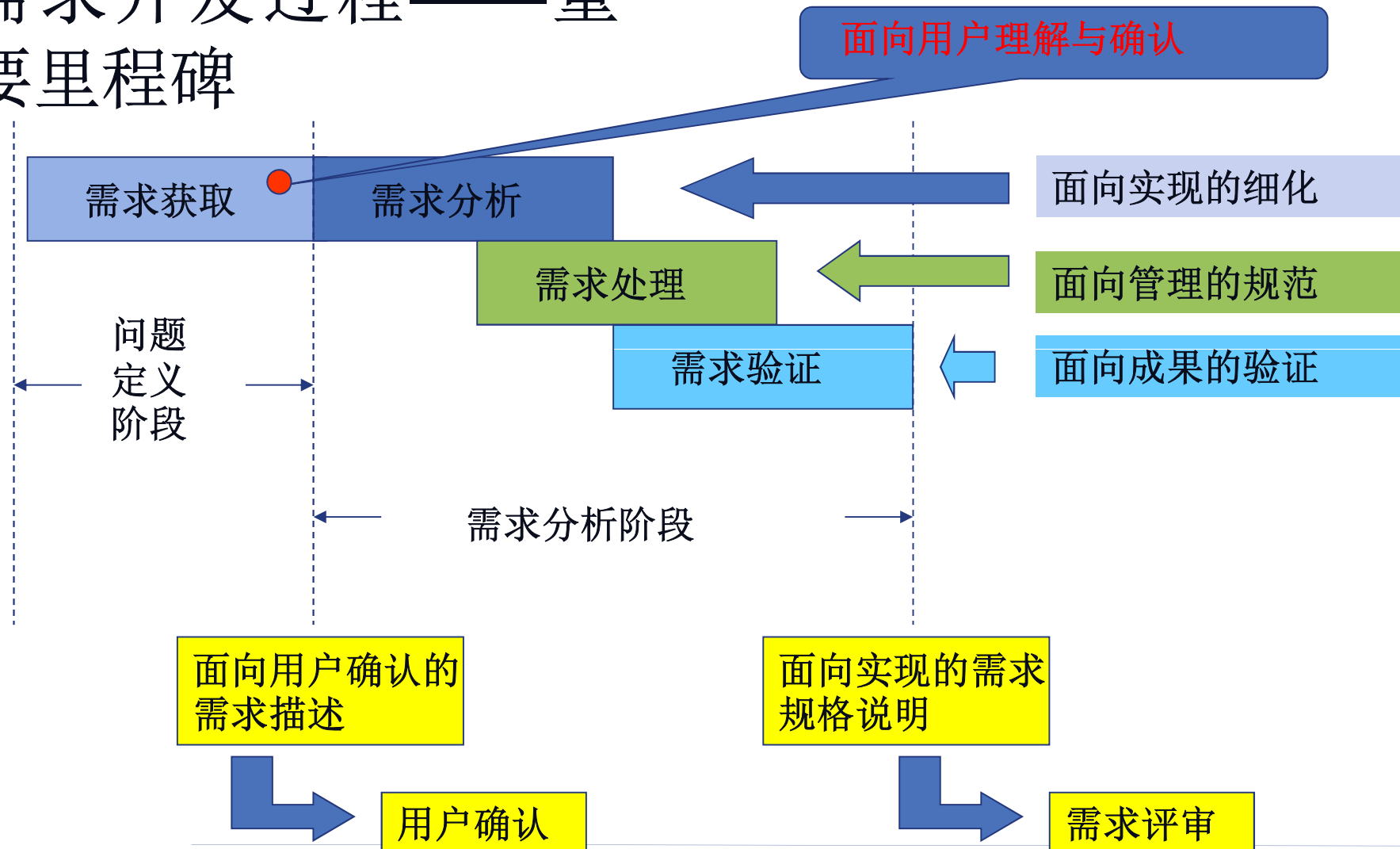
- 一致性
- 完整性
- 非歧义性

7. 管理

- 查找bug
- verification——软件是否按团队要求运行
- validation——软件是否按甲方要求运行
- 系统是否满足业务目标
- 向利益相关者提供进度

Requirements Engineering-III

需求开发过程——重要里程碑



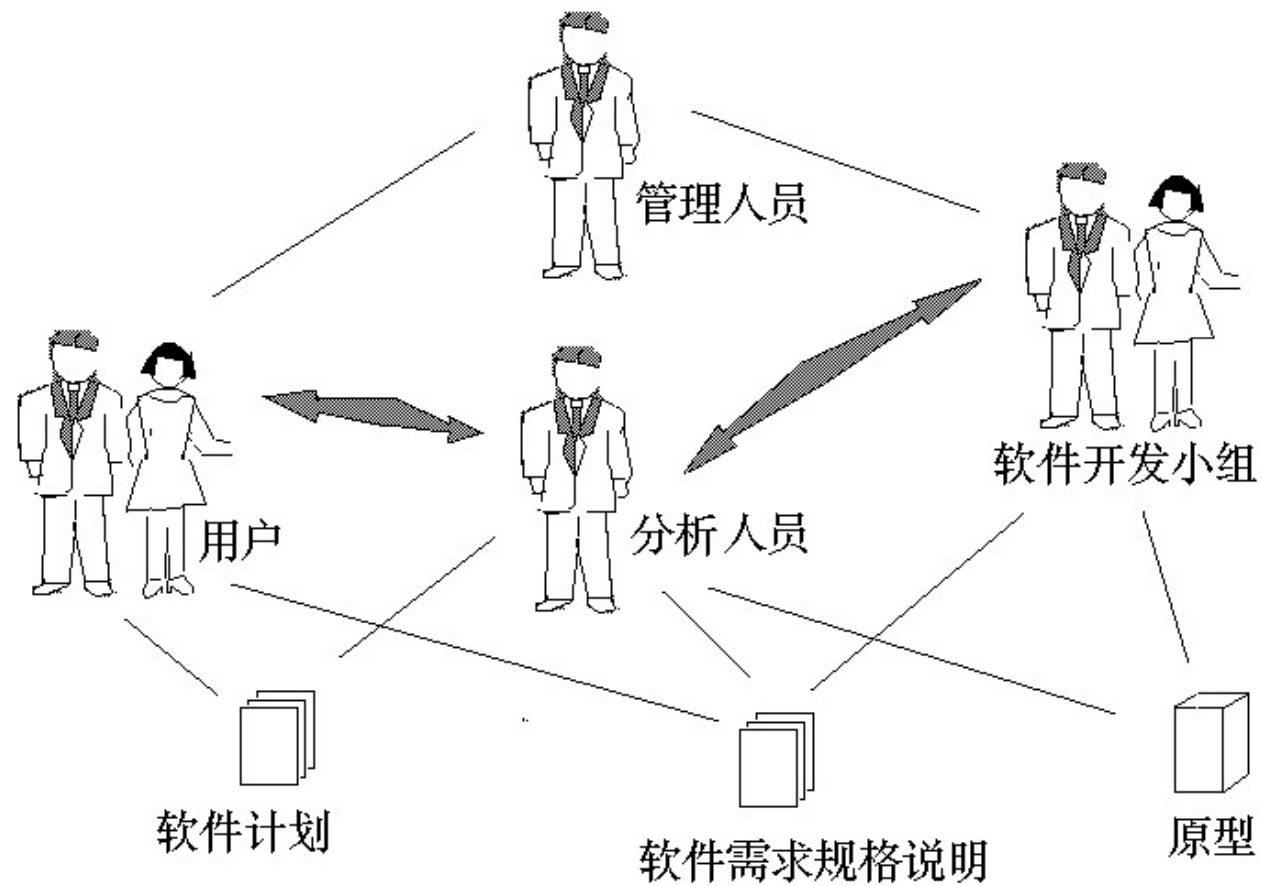
启动阶段

Inception

- ❖ **Identify stakeholders** 识别利益相关者
 - “who else do you think I should talk to?”
- ❖ **Recognize multiple points of view** 认识到多个观点
- ❖ **Work toward collaboration** 致力于协作
- ❖ **The first questions**
 - Who is behind the request for this work?
 - Who will use the solution?
 - What will be the economic benefit of a successful solution
 - Is there another source for the solution that you need?

Inception

❖ Identify stakeholders



需求获取

Eliciting Requirements

❖ Eliciting Principles

- meetings are conducted and attended by both software engineers and customers
- rules for preparation and participation are established
- an agenda is suggested
- a “facilitator (主持人)” (can be a customer, a developer, or an outsider) controls the meeting
- a "definition mechanism" (can be work sheets, flip charts, or wall stickers or an electronic bulletin board, chat room or virtual forum) is used

• 需求获取原则

- 会议由软件工程师和客户共同主持和参加
- 制定准备和参与的规则
- 建议议程
- 由“协调人”（可以是客户、开发人员或外部人员）控制会议
- 使用“定义机制”（可以是工作表、翻转图、墙贴或电子公告板、聊天室或虚拟论坛）

Eliciting Requirements

❖ The goal is

- to identify the problem
- propose elements of the solution
- negotiate different approaches, and
- specify a preliminary set of solution requirements

- 目标是

- 识别问题
- 提出解决方案的元素
- 协商不同的方法
- 指定初步的解决方案需求

启发工作产品

Elicitation Work Products

- ❖ a statement of need and feasibility.
- ❖ a bounded statement of scope for the system or product.
- ❖ a list of customers, users, and other stakeholders who participated in requirements elicitation
- ❖ a description of the system's technical environment.
- ❖ a list of requirements (preferably organized by function) and the domain constraints that apply to each.
- ❖ a set of usage scenarios that provide insight into the use of the system or product under different operating conditions.
- ❖ any prototypes developed to better define requirements.

- 需求获取工作产品

- 需求和可行性陈述
- 系统或产品范围的有界陈述
- 参与需求获取的客户、用户和其他利益相关者列表
- 系统技术环境的描述
- 需求列表（最好按功能组织）及适用于每个需求的领域约束
- 提供系统或产品在不同操作条件下使用的情景集合
- 用于更好定义需求的任何原型

质量功能部署 (QFD)

Quality Function Deployment

- ❖ **Function deployment** determines the “value” (as perceived by the customer) of each function required of the system
- ❖ **Information deployment** identifies data objects and events
- ❖ **Task deployment** examines the behavior of the system
- ❖ **Value analysis** determines the relative priority of requirements

Quality Function Deployment and others

3. 质量功能部署 (QFD) ☆

1. 功能部署——确定系统功能的价值
2. 信息部署——确定数据对象和事件
3. 任务部署——确定系统的行为
4. 价值分析——确定需求的优先级

4. 非功能需求 (NFR)

1. 质量属性
2. 性能属性
3. 安全属性
4. 其他约束

5. 用例 (Use-Cases) ——描述用户使用系统的场景

Non-Functional Requirements

❖ **Non-Functional Requirement (NFR)** – quality attribute, performance attribute, security attribute, or general system constraint. A two phase process is used to determine which NFR's are compatible:

- The first phase is to create a matrix using each NFR as a column heading and the system SE guidelines a row labels
 - The second phase is for the team to prioritize each NFR using a set of decision rules to decide which to implement by classifying each NFR and guideline pair as complementary, overlapping, conflicting, or independent
- 质量属性、性能属性、安全属性或一般系统约束。使用两阶段过程确定哪些NFR是兼容的：
 - 第一阶段是使用每个NFR作为列标题，系统SE指南作为行标签创建矩阵
 - 第二阶段是团队优先考虑每个NFR，使用一组决策规则决定实施哪些NFR，通过将每个NFR和指南对分类为互补、重叠、冲突或独立来实现

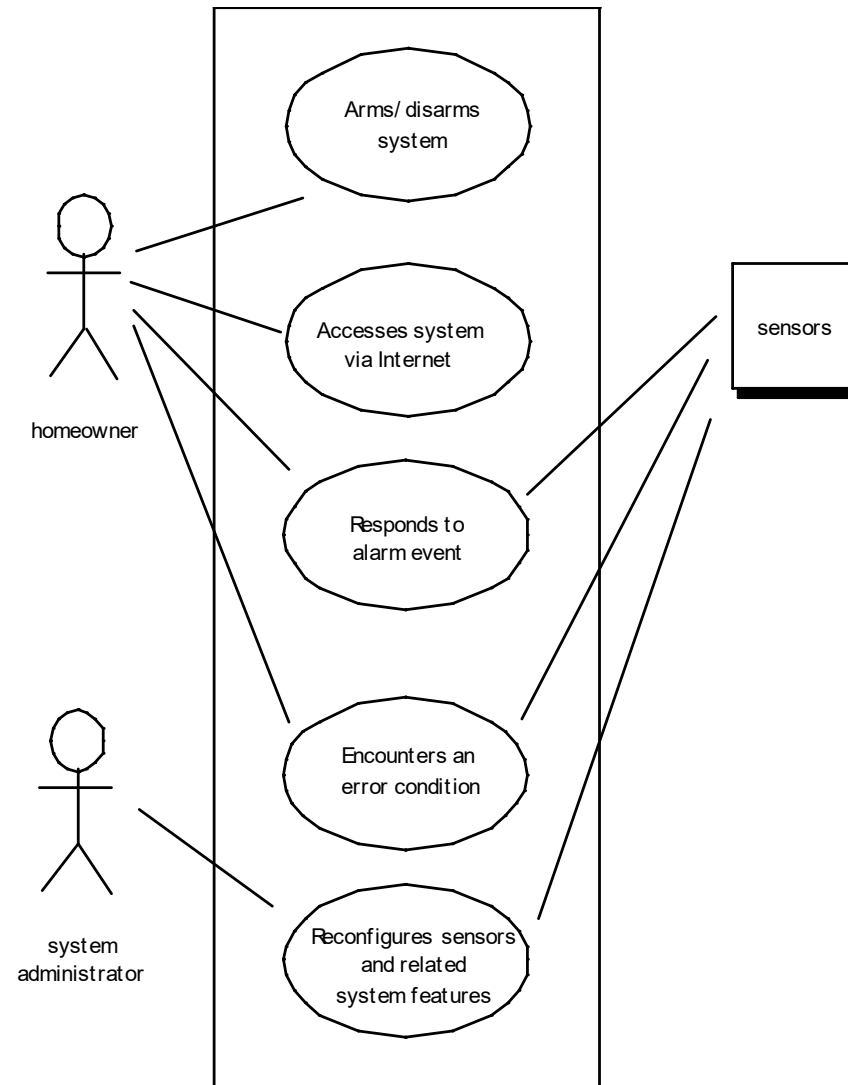
Use-Cases

- ❖ A collection of user scenarios that describe the thread of usage of a system
- ❖ Each scenario is described from the point-of-view of an “actor”—a person or device that interacts with the software in some way
- ❖ Each scenario answers the following questions:
 - Who is the primary actor, the secondary actor (s)?
 - What are the actor’s goals?
 - What preconditions should exist before the story begins?
 - What main tasks or functions are performed by the actor?
 - What extensions might be considered as the story is described?
 - What variations in the actor’s interaction are possible?
 - What system information will the actor acquire, produce, or change?
 - Will the actor have to inform the system about changes in the external environment?
 - What information does the actor desire from the system?
 - Does the actor wish to be informed about unexpected changes?

- 描述系统使用线程的用户情景集合

- 每个情景从“参与者”的角度描述——与软件互动的人员或设备

Use-Case Diagram



Elaboration: Building the Analysis Model

❖ Elements of the analysis model

■ Scenario-based elements

- Functional—processing narratives（描述） for software functions
- Use-case—descriptions of the interaction between an “actor” and the system

• 构建分析模型的要素

■ Class-based elements

- Implied by scenarios

• 基于情景的要素

• 功能性——软件功能的处理叙述（描述）

■ Behavioral elements

- State diagram

• 使用案例——描述“参与者”和系统之间的互动

• 基于类的要素

• 基于流的要素

■ Flow-oriented elements

- Data flow diagram

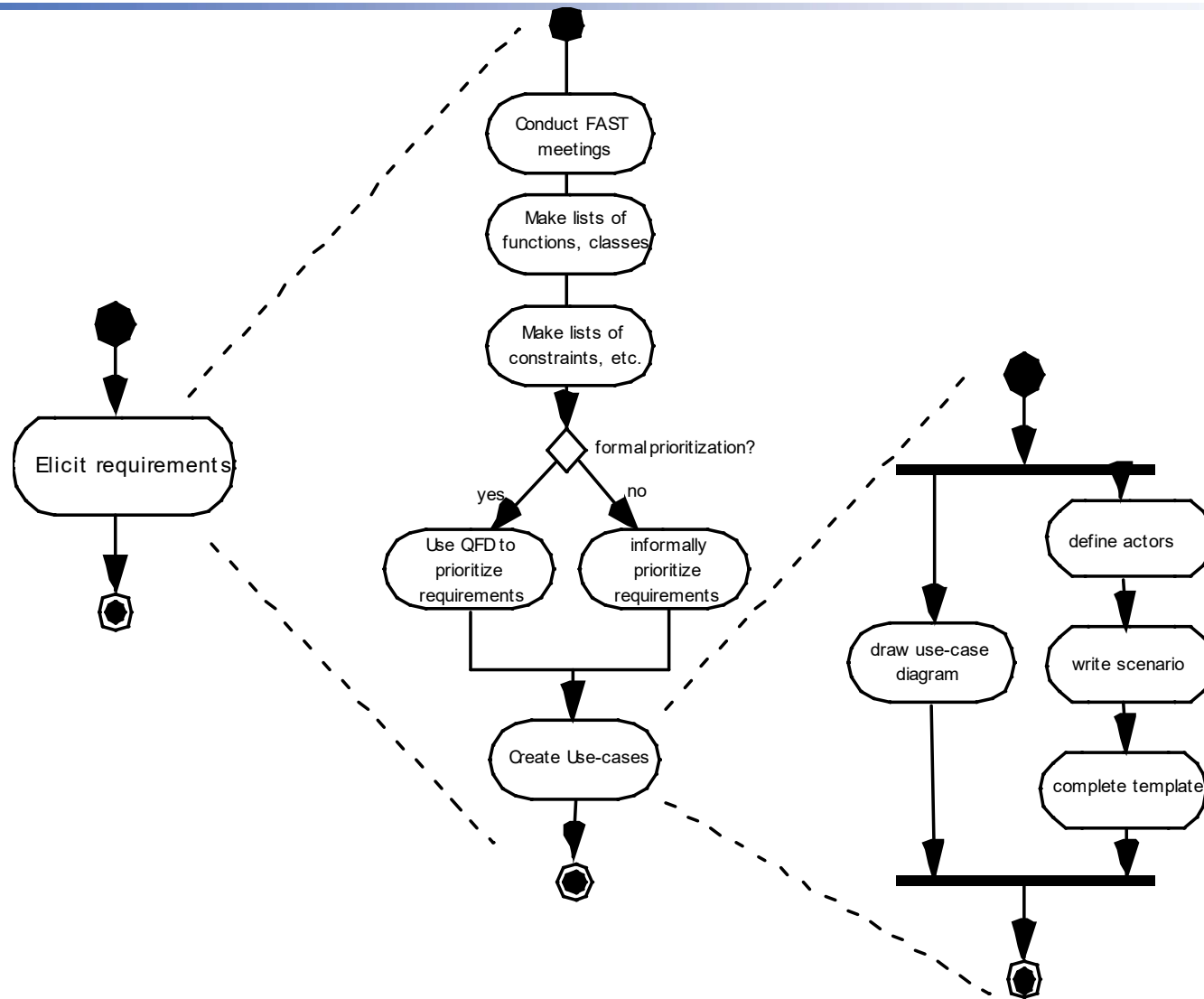
• 由情景暗示

• 数据流图

• 行为要素

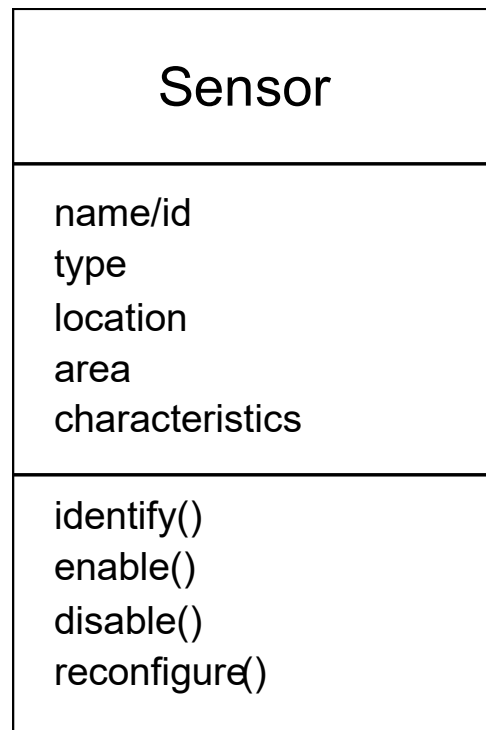
• 状态图

Activity Diagram

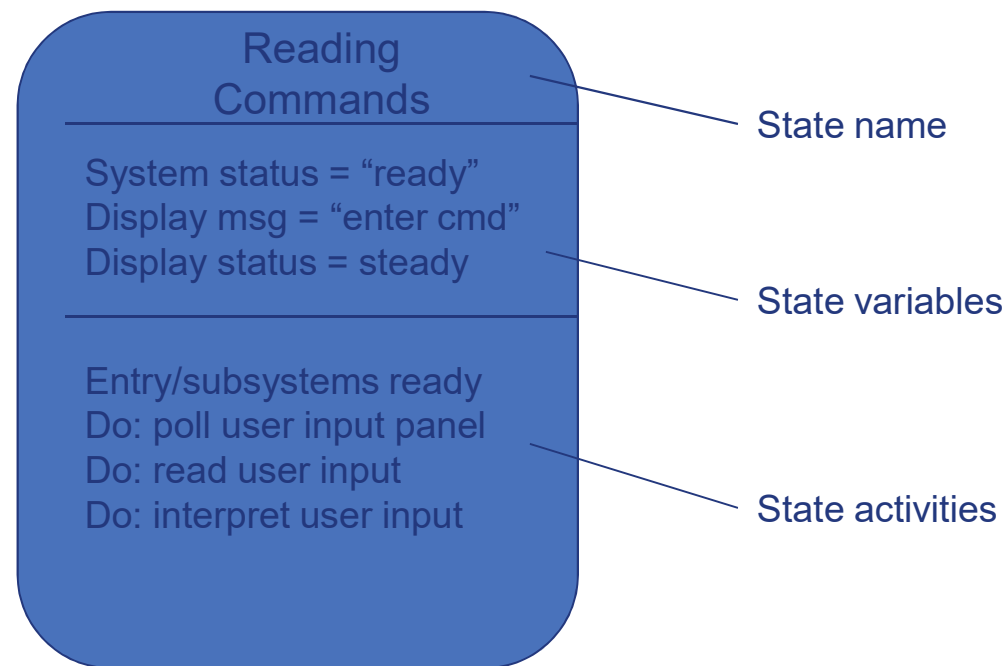


Class Diagram

From the *SafeHome* system ...



State Diagram



Analysis Patterns

Pattern name: A descriptor that captures the essence of the pattern.

Intent: Describes what the pattern accomplishes or represents

Motivation: A scenario that illustrates how the pattern can be used to address the problem.

Forces and context: A description of external issues (forces) that can affect how the pattern is used and also the external issues that will be resolved when the pattern is applied.

Solution: A description of how the pattern is applied to solve the problem with an emphasis on structural and behavioral issues.

Consequences: Addresses what happens when the pattern is applied and what trade-offs exist during its application.

Design: Discusses how the analysis pattern can be achieved through the use of known design patterns.

Known uses: Examples of uses within actual systems.

Related patterns: One or more analysis patterns that are related to the named pattern because (1) it is commonly used with the named pattern; (2) it is structurally similar to the named pattern; (3) it is a variation of the named pattern.

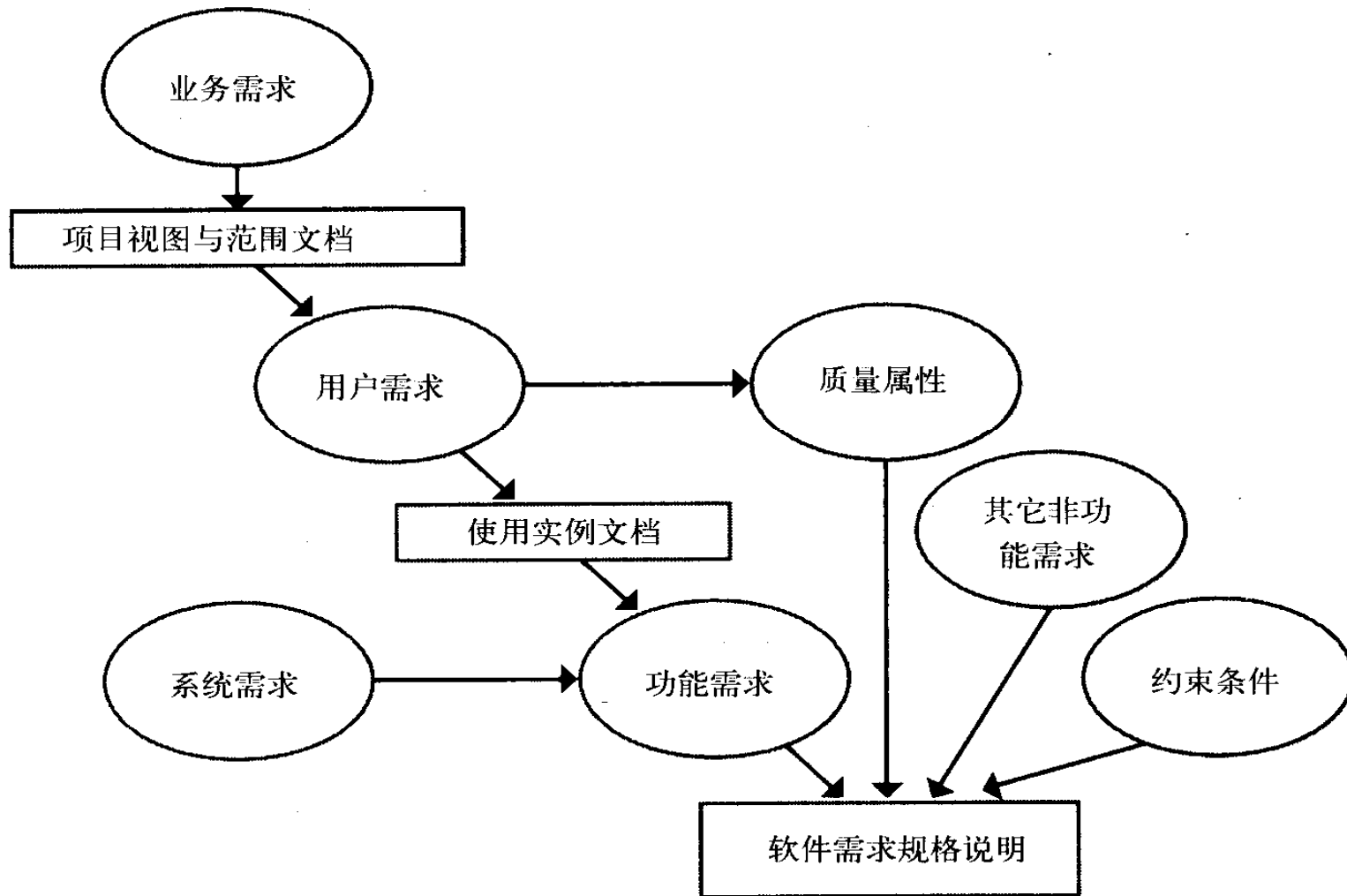
Analysis Patterns

- 模式名称：捕捉模式本质的描述符
- 意图：描述模式完成或表示的内容
- 动机：说明如何使用该模式解决问题的情景
- 力量和背景：描述外部问题（力量）如何影响模式的使用，以及应用模式时将解决的外部问题
- 解决方案：描述如何应用模式解决问题，重点在于结构和行为问题
- 后果：处理模式应用期间存在的内容及其权衡
- 设计：讨论如何通过已知设计模式实现分析模式
- 已知用途：实际系统中的用途示例
- 相关模式：描述与已知模式相关的一种或多种分析模式，原因包括（1）常与已知模式一起使用；（2）在结构上与已知模式相似；（3）是已知模式的变体

Negotiating Requirements

- ❖ **Identify the key stakeholders** 确定关键利益相关者
 - These are the people who will be involved in the negotiation 这些是将参与协商的人
- ❖ **Determine each of the stakeholders “win conditions”** 确定每个利益相关者的“获胜条件”
 - Win conditions are not always obvious 获胜条件并不总是显而易见
- ❖ **Negotiate** 协商
 - Work toward a set of requirements that lead to “win-win” 努力达成实现“共赢”的需求集合

Specification



软件需求各组成部分之间的关系

Specification

- ❖ **引言**：陈述软件目标，在基于计算机的系统语境内进行描述。
- ❖ **信息描述**：给出软件必须解决问题的详细描述，记录信息内容和关系、流和结构。
- ❖ **功能描述**：描述解决问题所需的每个功能。其中包括，为每个功能说明一个处理过程；叙述设计约束；叙述性能特征；用一个或多个图形来形象地表示软件的整体结构和软件功能与其他系统元素间的相互影响。
- ❖ **行为描述**：描述作为外部事件和内部产生的控制特征的软件操作。
- ❖ **检验标准**：描述检验系统成功的标志。即对系统进行什么样的测试，得到什么样的结果，就表示系统已经成功实现了。它是“确认测试”的基础。
- ❖ **参考书目**：包含了对所有和该软件相关的文档的引用，其中包括其他的软件工程文档、技术参考文献、厂商文献以及标准。
- ❖ **附录**：包含了规约的补充信息，表格数据、算法的详细描述、图表以及其他材料。

验证需求

Validating Requirements - I

- ❖ Is each **requirement consistent with the overall objective** for the system/product?
- ❖ Have all requirements been specified at the **proper level** of abstraction? That is, do some requirements provide a level of technical detail that is inappropriate at this stage?
- ❖ Is the requirement **really necessary** or does it represent an add-on feature that may not be essential to the objective of the system?
- ❖ Is each requirement **bounded** (有界限) and **unambiguous**?
- ❖ Does each requirement have **attribution** (归属)? That is, is a source (generally, a specific individual) noted for each requirement?
- ❖ Do any requirements **conflict with other requirements**?

Validating Requirements - II

- ❖ Is each requirement **achievable** in the technical environment that will house the system or product?
- ❖ Is each requirement **testable**, once implemented?
- ❖ Does the requirements model **properly reflect** the information, function and behavior of the system to be built.
- ❖ Has the requirements model been “**partitioned**” in a way that exposes progressively more detailed information about the system.
- ❖ Have **requirements patterns** been used to simplify the requirements model. Have all patterns been properly validated? Are all patterns consistent with customer requirements?

需求监控

Requirements Monitoring

Especially needs in incremental development

- *Distributed debugging* – uncovers errors and determines their cause.
- *Run-time verification* – determines whether software matches its specification.
- *Run-time validation* – assesses whether evolving software meets user goals.
- *Business activity monitoring* – evaluates whether a system satisfies business goals.
- *Evolution and co-design* – provides information to stakeholders as the system evolves.

特别需要在增量开发中

- 分布式调试——发现错误并确定其原因。
- 运行时验证——确定软件是否符合其规范。
- 运行时验证——评估演化中的软件是否符合用户目标。
- 业务活动监控——评估系统是否满足业务目标。
- 演化和协同设计——随着系统的发展向利益相关者提供信息。

Thank You !

lliao@seu.edu.cn